



Design and Implementation of a QR Code-Based Exam Attendance Management System

Prepared by:

Zakariya SABRI

Cybersecurity — First Year, Engineering Cycle

UM6P College of Computing, Rabat

Host Supervisor:

Mr. Abderrahmane SADIQ

Vice-Doyen chargé des affaires pédagogiques

Faculté Polydisciplinaire de Taroudant

Academic Supervisor:

Prof. Mohammed Amine KOULALI

Affiliate Professor

UM6P College of Computing

Taroudant

20 June — 20 August 2026

Academic Year: 2025–2026

Acknowledgements

I would like to thank Mr Abderrahmane Sadiq, Vice-Doyen chargé des affaires pédagogiques at the Faculté Polydisciplinaire de Taroudant, for hosting this internship, for explaining how examinations are organised at the faculty, and for reviewing the design of the system before it was built.

I would also like to thank Prof. Mohammed Amine Koulali, of the UM6P College of Computing, for supervising this work on the academic side.

My thanks go as well to the staff of the Service des affaires estudiantines, who answered my questions about the procedures followed during examination periods.

Finally, I thank my family and my fellow students for their support throughout this period.

Abstract

Attendance at examinations held at the Faculté Polydisciplinaire de Taroudant is recorded on a single printed sheet per room. Because only one copy exists, only one person can record attendance however many proctors are present, and when students do not occupy their assigned seats each name has to be searched for in a list of several dozen entries.

This report presents a system that removes that constraint. A server holds the attendance data and enforces every rule. A native Android application allows a proctor to record a student by scanning a code printed on the convocation the student already receives before each session. A web interface allows the administration to create examinations and to download the resulting attendance as a file. Several devices can therefore record the same examination at the same time, and the database itself guarantees that a student cannot be recorded twice.

The printed code is encrypted with AES in Galois/Counter Mode, so a code that the system did not issue cannot be used. Administrative operations require a signed token, and the administrator's password is stored only as a hash. A scan is accepted only while the examination is in progress. Every rule is applied by the server, so the correctness of a record does not depend on the device that produced it.

A secondary module examined the automatic verification of registration data by optical character recognition. It concludes that such reading can assist a human verifier but cannot replace one.

Keywords: QR code, examination attendance, REST API, Android, AES-GCM, JSON Web Token, optical character recognition.

Résumé

La présence aux examens organisés à la Faculté Polydisciplinaire de Taroudant est relevée sur une feuille imprimée unique par salle. Comme il n'en existe qu'un seul exemplaire, une seule personne peut relever la présence quel que soit le nombre de surveillants présents, et lorsque les étudiants ne s'installent pas à la place qui leur est assignée, chaque nom doit être recherché dans une liste de plusieurs dizaines d'entrées.

Ce rapport présente un système qui supprime cette contrainte. Un serveur conserve les données de présence et applique l'ensemble des règles. Une application Android native permet au surveillant d'enregistrer un étudiant en scannant un code imprimé sur la convocation que celui-ci reçoit déjà avant chaque session. Une interface web permet à l'administration de créer les examens et de télécharger la présence sous forme de fichier. Plusieurs appareils peuvent ainsi enregistrer le même examen simultanément, et la base de données garantit elle-même qu'un étudiant ne peut être enregistré deux fois.

Le code imprimé est chiffré avec AES en mode Galois/Counter, de sorte qu'un code non émis par le système ne peut pas être utilisé. Les opérations administratives exigent un jeton signé, et le mot de passe de l'administrateur n'est conservé que sous forme d'empreinte. Un scan n'est accepté que pendant le déroulement de l'examen. Toutes les règles sont appliquées par le serveur : l'exactitude d'un enregistrement ne dépend donc pas de l'appareil qui l'a produit.

Un module secondaire a étudié la vérification automatique des données d'inscription par reconnaissance optique de caractères. Il conclut qu'une telle lecture peut assister un vérificateur humain sans pouvoir s'y substituer.

Mots-clés : code QR, présence aux examens, API REST, Android, AES-GCM, jeton JSON Web Token, reconnaissance optique de caractères.

List of Abbreviations

AES	Advanced Encryption Standard
API	Application Programming Interface
CIN	Carte Nationale d'Identité
CNIE	Carte Nationale d'Identité Électronique
CSV	Comma-Separated Values
FIPS	Federal Information Processing Standard
FPT	Faculté Polydisciplinaire de Taroudant
GCM	Galois/Counter Mode
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ISO	International Organization for Standardization
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
NFC	Near Field Communication
NIST	National Institute of Standards and Technology
OCR	Optical Character Recognition
PFA	Projet de Fin d'Année
QR	Quick Response (code)
REST	Representational State Transfer
RFC	Request for Comments
RFID	Radio Frequency Identification
UM6P	Université Mohammed VI Polytechnique

Contents

Acknowledgements	i
Abstract	ii
Résumé	iii
List of Abbreviations	iv
General Introduction	1
1 Context and Objectives	2
1.1 The Host Institution	2
1.2 The Service des affaires estudiantines	2
1.3 Recording Attendance Today	3
1.4 Objectives of the Internship	4
1.5 Working Method	5
1.6 Organisation of the Report	5
2 Preliminary Study	7
2.1 Identifying a Student Without Searching a List	7
2.1.1 Manual entry	7
2.1.2 One-dimensional barcodes	7
2.1.3 Two-dimensional codes (QR)	8
2.1.4 Contactless cards and biometrics	8
2.1.5 Distribution: the practical constraint	8
2.2 Sharing Data Between Several Devices	9
2.2.1 The REST style	9
2.2.2 Separating the clients from the server	9
2.3 Restricting Access to Administrative Operations	9
2.3.1 Sessions and tokens	10
2.3.2 JSON Web Tokens	10
2.3.3 Storing passwords	10
2.4 Protecting Information Printed on Paper	11
2.4.1 The forgery problem	11
2.4.2 Signing	11
2.4.3 Encrypting	11

2.4.4	Key rotation	12
2.5	Optical Character Recognition	12
2.5.1	Principle	12
2.5.2	What degrades accuracy	12
2.5.3	Local engines and hosted services	13
2.6	Comparable Solutions	13
2.7	Summary	14
3	Requirements and Design	15
3.1	Functional Requirements	15
3.2	Non-Functional Requirements	17
3.3	Actors	17
3.4	Architecture	18
3.5	Technology Choices	20
3.5.1	Server	20
3.5.2	Database	20
3.5.3	Mobile application	20
3.5.4	Administration interface	21
3.5.5	Security choices	21
3.6	Data Model	22
3.7	Recording a Scan	24
3.8	Summary	25
4	Implementation	26
4.1	The Server	26
4.1.1	Organisation of the code	26
4.1.2	Entities	26
4.1.3	Objects exchanged with the clients	27
4.1.4	Interface	27
4.1.5	Validation and errors	28
4.1.6	Recording a scan	29
4.2	Security	29
4.2.1	Protecting the printed code	29
4.2.2	Authenticating the administrator	30
4.2.3	Why the mobile application has no account	30
4.3	The Mobile Application	31
4.3.1	Choosing an exam block	33
4.3.2	Scanning	33
4.4	The Administration Interface	34
4.5	Producing the Attendance File	35

4.6	The Verification Module	36
4.6.1	What was built	36
4.6.2	What the documents showed	36
4.6.3	Three outcomes rather than two	36
4.6.4	Why a more accurate engine was not used	37
4.6.5	What the module is for	37
4.7	Testing	37
4.8	Summary	38
5	Results, Limitations and Future Work	40
5.1	What the System Does	40
5.2	Limitations	41
5.2.1	Deliberate boundaries	41
5.2.2	Limits imposed by the conditions of the internship	41
5.2.3	Defects and rough edges	42
5.2.4	The verification module	43
5.3	Future Work	43
5.4	Summary	44
	General Conclusion	45
	Bibliography	47
A	Interface Contract	49
A.1	Operations	49
A.2	Authentication	49
A.3	Error Format	50
A.4	Obtaining a Printed Code	51
A.5	Recording a Scan	51
A.6	Creating an Exam Block	52
A.7	Downloading the Attendance	52

List of Figures

3.1	Use case diagram of the system	18
3.2	Architecture of the system	19
3.3	The convocation, in the format proposed for this system, with the encrypted code printed in the upper right corner	22
3.4	Entity relationship diagram of the system	23
3.5	Sequence diagram: recording a student	24
4.1	The exam block list on the proctor's telephone	32
4.2	The list of exam blocks in the administration interface	34

List of Tables

3.1	Functional requirements of the administration interface	15
3.2	Functional requirements of the mobile application	16
3.3	Rules enforced by the server	16
3.4	Non-functional requirements	17
3.5	Technologies used and their role	20
4.1	Operations exposed by the server	27
4.2	Representative verifications carried out	38
5.1	Work required before the system could be used	43
A.1	The complete set of operations	49
A.2	Error responses	50
A.3	Refusals when recording a scan	51

General Introduction

A large part of what a university does has nothing to do with teaching. Students have to be registered, their records kept and corrected, examinations scheduled, rooms assigned, documents printed and distributed, and attendance recorded and archived. These procedures attract little attention as long as they work. They consume staff time all the same, and a procedure designed when an institution was small tends to become expensive as the institution grows, without anyone deciding that it should.

The Faculté Polydisciplinaire de Taroudant is an institution of that kind. It opened in 2010 with a modest number of students, and its enrolment has risen steadily since. Some of its administrative procedures have not changed at the same rate, and examination periods are when the difference is felt most: many rooms, many students, and a limited number of staff available to supervise them.

This internship examined two such procedures. The first is the recording of attendance during examinations, which today depends on a single printed sheet held by one person in each room. The second is the verification of the information a student enters when registering, which is compared with their identity documents by hand. Both are ordinary administrative tasks. Both are slow for reasons that have more to do with the form the information takes than with the work itself.

The two proved to be very different problems. Recording attendance is a question of organisation: the information needed already exists, and the difficulty is that it is held in a place where only one person can reach it at a time. Verifying a registration is a question of reading, and reading a photograph of an identity document turns out to be considerably harder than it appears, for reasons that are partly technical and partly legal. The first problem admitted a complete solution within the time available. The second produced a result of a different kind: a working system, a clear account of what it cannot do, and an argument about the role such a system should be given.

This report describes the design and construction of a system that records examination attendance by reading a code printed on the document each student already receives, and it presents the study carried out on the automatic verification of registration data. It sets out the decisions that shaped both, the reasons behind them, and the limits of what was achieved. Rather than presenting only the parts that worked, it also states where the system falls short and what would be required before the faculty could rely on it.

Context and Objectives

This chapter describes the environment in which the internship took place and the problem it set out to solve. It presents the host institution and the department that hosted the work, explains how attendance is currently recorded during examinations, states the objectives that were set, and describes the working method followed over the two months of the internship. It closes with the organisation of the rest of this report.

1.1 The Host Institution

This internship took place at the Faculté Polydisciplinaire de Taroudant, a public higher-education institution attached to Université Ibn Zohr in Agadir. The faculty was created by decree in 2006 and opened in 2010. Its purpose was to bring university education closer to students in the south of Morocco, so that young people from the region would no longer have to move to Agadir to begin their studies.

The faculty teaches in three broad fields: Sciences and Techniques, Economics and Management, and Letters and Human and Social Sciences. It remains a small institution in terms of staff. Its student population, however, has continued to grow, following the rising number of baccalaureate holders in the regions served by the university.

That growth is directly relevant to this project. Licence programmes in particular enrol large groups, and examinations are held in rooms that hold several dozen students. Procedures that were designed when the faculty was young come under real pressure during examination periods, and the recording of attendance is one of them.

1.2 The Service des affaires estudiantines

The internship was carried out within the Service des affaires estudiantines, the department responsible for the administrative side of student life. Its work covers registration, student records, the organisation of examinations, and the documents issued to students, including the *convocation* that each student receives before an examination session.

This service was the natural place to study the problem addressed in this report. It produces the documents used before and during examinations, and it is also the service that experiences the consequences when those documents make a task slower than it needs to be.

The work was supervised by Mr Abderrahmane Sadiq, Vice-Doyen chargé des affaires pédagogiques, who defined the objectives, reviewed the design at several stages, and validated

the final result.

1.3 Recording Attendance Today

Before each examination session the administration prepares, for every examination room, a single printed sheet listing the students assigned to that room. The sheet gives each student's table number, name and administrative information, and the rows are ordered by table number. During the examination the proctor goes through the room and records who is present.

Alongside this sheet, every student receives a personal document called a *convocation*. It lists the examinations the student must sit, with the date, time and room for each one, and it carries a photograph of the student.

The procedure asks the proctor to perform two distinct tasks for every student in the room. The first is to confirm that the person sitting at the table really is that student, by comparing the face with the photograph on the convocation, or with the student card or the national identity card when no photograph is available. The second is to find that student's name in the list and mark them present.

The first task is a genuine control and it works. The difficulty lies entirely in the second.

If every student sits at the table assigned to them, the second task is easy: the list is ordered by table number, so the proctor moves down the room and down the page at the same time. In practice this often does not happen. Some students simply sit elsewhere. Others intend to respect their assigned place but cannot, because the table is broken or its number has worn away and can no longer be read. Once the seating order is broken the list can no longer be followed in order, and the proctor has to search for each name in a list that may contain several dozen entries.

A second difficulty is more serious, because it does not depend on the students at all. There is only one sheet per room. Attendance is therefore recorded by one person, however many proctors are present. The task cannot be divided, and putting more staff in the room does not make it faster.

This matters because of who those proctors are. The licence programmes enrol very large numbers of students, and the faculty does not have enough permanent staff to supervise every room. Master's and doctoral students are regularly asked to help. They are present for a single session, they do not know the students, and searching an unfamiliar list of names is slower for them than for a teacher who recognises the group.

Recording attendance is not in itself a difficult task. What makes it costly is that it has been tied to a single physical document. Any improvement therefore has to remove that constraint. It must allow several people to record attendance at the same time, it must make it possible to identify a student without reading through a list of names, and it must leave the verification of identity exactly where it is today, with the proctor.

1.4 Objectives of the Internship

The internship agreement defines the subject in the following terms:

Développement d'une application de vérification des données des étudiants par OCR lors de l'inscription administrative et d'un module de gestion des absences par QR Code.

That is, the development of an application that verifies student data through optical character recognition at the time of administrative registration, together with a module for managing absences based on QR codes. The agreement lists the associated missions: analysing the functional requirements, designing the database and the application architecture, developing a QR-based attendance module, generating and reading QR codes to identify students and record their presence automatically, developing an OCR module able to extract information from the national identity card, and comparing the extracted information automatically with the data entered at registration.

Two modules were therefore expected. They address different problems, and the work devoted to each was not equal. The attendance module became the main subject of the internship: the need was clearly expressed by the department, the problem was well defined, and a complete system could realistically be built and validated within the available time. The verification module was treated as a second, smaller study. As Chapter 4 explains, the constraints that apply to the automatic reading of identity documents are severe, and the conclusions drawn from that study are of a different nature from those of the attendance system.

For the attendance system, the objectives followed directly from the difficulties described above:

- to allow attendance to be recorded by several people at the same time, rather than by one person holding one document;
- to make it possible to record a student without searching for their name in a list;
- to leave the verification of identity where it is today, as a check carried out by the proctor;
- to produce, at the end of a session, a record that the administration can keep and use;
- to protect both the data and the operations that modify it.

A remark on terminology is needed here. The agreement speaks of managing absences, while the system described in this report records presence: a student who is scanned is present, and absence is what remains once the session is closed. The two are equivalent, but presence is what is actually observed in the examination room, and this report uses that term throughout.

Finally, although the subject belongs to software engineering, this internship forms part of a cybersecurity curriculum, and the security of the system was treated as an objective in its

own right rather than as a final adjustment. The information handled is personal, the operation recorded has academic consequences for the student, and the QR codes are distributed on paper, outside any controlled environment. Encryption of the QR payload, authentication of the administrative interface, hashing of stored credentials and restriction of the operations exposed by the server were therefore part of the work itself. They are described in Chapter 4.

1.5 Working Method

The internship ran from 20 June to 20 August 2026 and was carried out in a hybrid arrangement agreed in the internship agreement. On-site sessions at the faculty were used for follow-up meetings, for the design phases and for demonstrations. Development, testing and the writing of documentation were carried out remotely, with regular progress meetings.

The work followed a simple order that was kept throughout. Requirements were gathered first, by discussing with the supervisor how examinations are actually organised and what the department needed from a new system. The design of the data model and of the overall architecture came next, and was reviewed with the supervisor before implementation began. The server was then built and tested on its own, before either of the applications that use it. The mobile application and the administration interface were developed afterwards, against a server whose behaviour was already fixed and documented.

This order was deliberate. The two client applications communicate only with the server and never with each other, so any change to the server after they were written would have required changing both. Building and stabilising the server first meant that each client could be written once, against a description of the interface that did not move underneath it.

Testing was manual and continuous rather than concentrated at the end. Each part of the system was exercised as soon as it was written, first through direct requests to the server, then through the applications themselves, and finally on a real Android device rather than an emulator. Version control was used throughout, with the work committed in small steps so that any change could be examined or reversed on its own.

1.6 Organisation of the Report

This report is organised in five chapters.

Chapter 2 presents the preliminary study carried out before any design decision was taken. It covers QR codes and their use for identification, the separation between a server and its clients in a REST architecture, the approaches available for authenticating an administrative interface, symmetric encryption and the guarantees it provides, and the principles and known limits of optical character recognition. It closes with a look at comparable solutions.

Chapter 3 sets out the requirements and the design. It states the functional and non-functional requirements, identifies the actors, presents the architecture of the system and the technologies

chosen with the reasons for each, and describes the data model as it was validated with the supervisor.

Chapter 4 describes the implementation: the server and its interface, the security mechanisms, the mobile application used by proctors, the administration interface, the export of attendance records, the secondary study on optical character recognition, and the testing carried out.

Chapter 5 presents the results obtained, states the limitations of the system, and describes the work that would be required before it could be used in production.

Preliminary Study

Before designing anything, it was necessary to understand the techniques that could be used and the constraints attached to each of them. This chapter presents that study. It begins with the ways a student can be identified quickly in an examination room, then examines how several devices can share a single set of data, how access to an administrative interface can be restricted, and how information printed on paper can be protected against forgery. It then presents the principles and the known limits of optical character recognition, which concern the second module. The chapter closes with a look at how similar problems have been solved elsewhere.

This chapter describes possibilities rather than decisions. The choices made for this project, and the reasons behind them, are presented in Chapter 3.

2.1 Identifying a Student Without Searching a List

The central difficulty described in Chapter 1 is that finding a student in a printed list is slow. Several technologies can replace that search, and they differ in cost, in the equipment they require, and in the way the identifier reaches the student.

2.1.1 Manual entry

The simplest approach is to have the proctor type the student's registration number into an application. It requires no special equipment beyond a device with a keyboard. It is also the slowest option, and it introduces a new kind of error: a mistyped digit records the wrong student as present, and nothing in the process would reveal the mistake. For a room of several dozen students, typing is not a serious improvement over reading a list.

2.1.2 One-dimensional barcodes

A conventional barcode encodes a short numeric value in a series of parallel bars. It is cheap to print and can be read by a dedicated scanner or by a camera. Its limitations are physical: it stores little information, it can only be read from one orientation, and a barcode that is creased or partly obscured usually becomes unreadable. Since the codes in this project would be printed on paper carried by students, the tolerance to damage matters.

2.1.3 Two-dimensional codes (QR)

A QR code stores data in a square grid of black and white modules, and is read in two dimensions rather than one. The format was created by Denso Wave in 1994 and is standardised as ISO/IEC 18004.

Two properties make it well suited to a printed document. The first is orientation: three large squares in the corners allow a reader to determine the position and rotation of the code, so it can be scanned from any angle. The second is error correction. A QR code is encoded with redundant data using the Reed–Solomon algorithm, at one of four levels, conventionally named L, M, Q and H, which allow roughly seven, fifteen, twenty-five and thirty percent of the code to be recovered when damaged. A code that has been folded, stained or partly torn can therefore still be read.

Reading requires only a camera, which every modern smartphone already has. No dedicated hardware is needed, and no equipment has to be bought for each examination room.

2.1.4 Contactless cards and biometrics

Two other approaches deserve mention. Contactless cards, using RFID or NFC, identify a student without any visual reading at all, and are used by some universities for building access. They require a card to be issued to every student and a reader in every room, which represents a real cost and an ongoing process for replacing lost cards.

Biometric identification, most often by fingerprint, removes the need for the student to carry anything. It is the most expensive option, it requires a terminal in each room, and it involves collecting and storing biometric data, which is a category of personal data subject to particular legal protection. For a faculty that already identifies students visually through a photograph, the additional cost and the additional legal exposure are difficult to justify.

2.1.5 Distribution: the practical constraint

Whichever technology is chosen, every student must receive their identifier. This is often the part that decides feasibility. A card system requires an issuing process, a replacement process, and a way to handle students who forget their card. A printed code requires only that the code appear on a document the student already receives.

The faculty already prints and distributes a personal *convocation* to each student before every examination session. That document already carries the student's photograph and the details of their examinations. Adding a printed code to it costs nothing beyond the change to the document template, and it means the code is renewed at every session rather than issued once and kept for years.

2.2 Sharing Data Between Several Devices

The second requirement from Chapter 1 is that several people should be able to record attendance at the same time. This is not a question of the reading technology but of where the data lives. If each device kept its own record, the results would have to be merged afterwards, and two devices recording the same student would produce a conflict that nothing could resolve automatically.

The usual answer is a central server that holds the data, with the devices acting as clients that read from it and write to it. The server becomes the single source of truth, and a rule such as “a student can only be recorded once for a given examination” can be enforced in one place instead of on every device.

2.2.1 The REST style

REST is an architectural style for such systems, described by Roy Fielding in 2000. It organises a server around resources, each identified by an address, and manipulated through the standard methods of the HTTP protocol: retrieving a resource, creating one, replacing one, deleting one. Data is commonly exchanged in JSON, a text format that is readable by a human and supported by every mainstream programming language.

One property of the style matters for what follows. A REST interface is stateless: each request carries everything the server needs to process it, and the server keeps no memory of previous requests from the same client. This makes the server simpler, allows several instances of it to run without sharing memory, and directly affects how authentication can be designed.

2.2.2 Separating the clients from the server

A further consequence is that different clients can use the same server. A mobile application and a web page are written in different languages and run in different environments, but if both speak HTTP and JSON, both can use the same interface without the server knowing or caring which is which.

This separation has a cost. The interface between them becomes a contract, and any change to it affects every client at once. Where the clients are developed separately, that contract has to be written down precisely, or the clients will be built against assumptions rather than facts.

2.3 Restricting Access to Administrative Operations

Not every operation in such a system can be left open. Reading the list of examination sessions is harmless. Creating, modifying or deleting one is not, and neither is downloading the attendance of a session, which contains personal data. Those operations must be reserved for an identified administrator.

2.3.1 Sessions and tokens

There are two common ways to keep a user authenticated after they have logged in.

In the session approach, the server creates a record when the user logs in, stores it in memory or in a database, and gives the client an identifier, usually in a cookie. Every later request carries that identifier and the server looks the session up. The server holds the state, which makes revoking a session immediate: the record is deleted and the identifier stops working.

In the token approach, the server issues a signed document containing the identity of the user and an expiry time. The client sends that document with every request. The server verifies the signature and reads the contents, without needing to remember anything. The state lives in the token rather than on the server.

The trade-off is symmetrical. Sessions are easy to revoke but require the server to keep state, which conflicts with the stateless character of a REST interface. Tokens fit that interface naturally but cannot be cancelled before they expire, since the server holds no record of them.

2.3.2 JSON Web Tokens

The most widely used token format is the JSON Web Token, defined in RFC 7519. It consists of three parts separated by dots: a header describing the algorithm used, a payload containing the claims, such as the subject and the expiry time, and a signature over the first two parts.

An important point is often misunderstood. The payload of a JWT is encoded, not encrypted. Anyone holding the token can read its contents. What the signature guarantees is that the contents have not been altered, because any modification invalidates the signature. Confidential information must therefore never be placed in a token.

The signature can be produced in two ways. A symmetric algorithm such as HS256 uses one secret key, held by the server, for both signing and verification. An asymmetric algorithm such as RS256 uses a private key to sign and a public key to verify, which allows a service to verify tokens it could not have issued. The asymmetric approach is valuable when issuance and verification are performed by different parties, and adds key management work when they are not.

2.3.3 Storing passwords

Whatever the mechanism, an administrator has a password, and that password has to be stored somewhere. Storing it in clear text is indefensible: anyone who obtains the file obtains the password. Storing a simple hash of it, using a fast function such as SHA-256, is better but still weak, because a fast function allows an attacker to test billions of candidate passwords per second.

Algorithms designed for passwords take the opposite approach. `bcrypt`, published by Provos and Mazières in 1999, is deliberately slow and includes a configurable cost factor that can be

raised as computers become faster. It also generates a random salt for each password, so two identical passwords produce different stored values and precomputed tables are useless. Verification consists of hashing the submitted password with the stored parameters and comparing the results, so the original password is never recovered, only matched.

2.4 Protecting Information Printed on Paper

A code printed on a document leaves the control of the institution the moment it is handed to a student. Anyone can look at it, photograph it, or attempt to create their own.

2.4.1 The forgery problem

If a QR code contains a student's registration number in plain text, that number is visible to anyone who scans the code with an ordinary telephone. More seriously, anyone can produce a code containing any number they choose, using one of the many free generators available online. A system that accepts whatever the code contains has no way of telling a code it issued from a code invented five minutes earlier.

Two families of solution exist, and they answer different questions.

2.4.2 Signing

A message authentication code, such as HMAC, attaches a short value computed from the content and a secret key. Anyone can read the content, but only the holder of the key can produce a valid authentication value for it. This proves that the code was issued by the system and has not been altered. It does not hide the content.

2.4.3 Encrypting

Encryption transforms the content into a form that cannot be read without the key. In symmetric encryption the same key encrypts and decrypts, which is appropriate when a single party performs both operations. The Advanced Encryption Standard, standardised as FIPS 197, is the algorithm in general use, and is accelerated in hardware on modern processors.

AES operates on fixed-size blocks, and the mode of operation determines how blocks are combined. The choice of mode matters more than is sometimes assumed. In ECB mode, identical plaintext blocks always produce identical ciphertext blocks, which reveals structure and makes the mode unsuitable for anything but the shortest values. CBC mode chains the blocks together and requires an initialisation vector, but on its own provides no protection against an attacker modifying the ciphertext.

Galois/Counter Mode, described in NIST SP 800-38D, belongs to a family known as authenticated encryption. It produces, in addition to the ciphertext, an authentication tag computed

over the whole message. Decryption verifies that tag and fails if the message has been altered by even one bit. In other words, it provides confidentiality and integrity in a single operation, which is what signing and encrypting separately would otherwise require.

GCM has one requirement that must be respected: the initialisation vector must never be reused with the same key. A fresh random value for every encryption satisfies this condition, and the value is not secret, so it can be transmitted alongside the ciphertext.

2.4.4 Key rotation

Any scheme based on a secret key must consider what happens when that key is replaced, whether because it has been exposed or as ordinary precaution. Codes issued under the old key stop being valid. Where codes are printed once and kept for years, rotating a key means reissuing every code in circulation. Where codes are printed for each occasion, the problem does not arise: the next printing simply uses the new key.

2.5 Optical Character Recognition

The second module of the internship concerns the automatic reading of identity documents. This section presents the technique and, more importantly, its limits, since those limits determine what such a module can honestly be expected to do.

2.5.1 Principle

Optical character recognition converts an image containing text into machine-readable characters. A typical pipeline begins with preprocessing, which may correct the orientation of the image, convert it to black and white, and reduce noise. It then segments the image into regions, lines and individual characters, and finally recognises each character. Modern engines perform recognition with neural networks trained on large collections of text images.

Tesseract, originally developed at Hewlett-Packard in the 1980s, released as open source in 2005 and since maintained with the support of Google, is the most widely used free engine. It runs locally, supports many languages through downloadable trained models, and exposes parameters controlling how the page is segmented.

2.5.2 What degrades accuracy

Published accuracy figures for OCR are usually measured on scanned printed documents: straight, evenly lit, high resolution, plain background. A photograph of an identity card taken with a telephone satisfies none of these conditions.

Resolution and focus limit what can be distinguished at all. Uneven lighting and reflections, which are common on a laminated card, remove parts of characters. Rotation is a particular

difficulty, since an engine that expects horizontal text will produce meaningless output from an image that is merely sideways, without reporting any error. Decorative backgrounds and security patterns, which identity documents carry deliberately, interfere with the separation of text from background. Documents printed in more than one script, as Moroccan identity documents are, add a further difficulty when the engine is configured for only one of them.

The consequence is not that OCR fails, but that it fails unevenly. Some fields are read correctly, others are read incorrectly, and others produce nothing usable. A system built on top of OCR must therefore be able to represent the case where a field could not be read at all, and must not confuse it with the case where a field was read and does not match.

2.5.3 Local engines and hosted services

Commercial services offered by cloud providers are considerably more accurate than a locally installed engine, particularly on photographs. Using one means transmitting the image to a third party.

For an identity document belonging to a student, that transmission is the whole question. The image contains personal data, and sending it outside the institution to a service whose processing and retention cannot be verified is not something a university can do casually. This constraint is not technical, and no improvement in accuracy removes it. It restricts the realistic options to engines that run on the institution's own machines.

2.6 Comparable Solutions

Attendance systems based on QR codes are not new, and it is worth being precise about how the published ones differ from what this project required.

The common arrangement, used by a number of classroom attendance applications, works in the opposite direction from the one needed here. The teacher displays a QR code on a screen, often regenerated every few seconds, and each student scans it with their own telephone to register their presence. This suits a lecture hall: no equipment is needed beyond what the students already carry, and the recording is instantaneous for a large group.

That arrangement cannot be used in an examination. Telephones are not permitted in the room, so the students cannot scan anything. The direction has to be reversed: the students carry the codes on paper, and the proctor holds the only device.

The reversal has an interesting consequence. The classroom arrangement has a well-known weakness, in that a student who is present can photograph the displayed code and send it to an absent friend, which is why such systems regenerate the code frequently and sometimes check the location of the device. When the proctor holds the only device and the student must be physically present with their document, that weakness disappears, and the identity check remains what it has always been: a person comparing a face with a photograph.

Other approaches are in use in higher education. Contactless student cards are common in institutions that already use them for library or building access, where the reader infrastructure exists. Biometric terminals are used in some examination contexts, at significantly higher cost. Both were considered in the light of the distribution question discussed earlier: they require equipment in every room and a card or an enrolment process for every student, whereas a printed code requires a change to a document that is already produced.

2.7 Summary

This study leads to a small number of conclusions that shape the next chapter.

Identification by QR code is the option that fits the constraints of the faculty: it needs no equipment beyond a telephone camera, it tolerates the damage a printed document suffers in practice, and it can be distributed on a document that already exists. Sharing the data between several devices requires a central server, and a REST interface allows a mobile application and a web interface to use the same one. Restricting administrative operations requires authentication, and the choice between sessions and tokens interacts with the stateless nature of that interface. Optical character recognition is limited both by the quality of photographs of identity documents and by the impossibility of sending those documents to an external service.

Chapter 3 uses these conclusions to state the requirements of the system precisely and to justify each of the choices made.

Requirements and Design

This chapter turns the problem described in Chapter 1 into a specification, and the possibilities examined in Chapter 2 into decisions. It begins with the functional and non-functional requirements of the system, identifies the people who use it, and presents the overall architecture. It then justifies each technology chosen, describes the data model as it was validated with the supervisor, and follows a single scan through the system from the camera to the database.

3.1 Functional Requirements

The requirements below follow directly from the objectives stated in Chapter 1. They are separated according to who performs the operation, since the two users of the system have very different roles and very different levels of trust.

Table 3.1: Functional requirements of the administration interface

Ref.	Requirement
FR-A1	The administrator must authenticate before performing any operation that modifies data or exposes attendance records.
FR-A2	The administrator can create an examination session, specifying the module, the type of session, the date and time, the duration and the academic year.
FR-A3	The administrator can modify an existing examination session.
FR-A4	The administrator can delete an examination session. Deletion must be refused when attendance records already exist for it, unless deletion is explicitly confirmed.
FR-A5	The administrator can view the list of examination sessions.
FR-A6	The administrator can download the attendance of a session as a file that can be opened, printed and archived.
FR-A7	The administrator can obtain the code to be printed on a student's convocation.

Table 3.2: Functional requirements of the mobile application

Ref.	Requirement
FR-M1	The proctor can view the list of examination sessions and search it by module name.
FR-M2	The proctor selects one session and records attendance for that session only.
FR-M3	The proctor records a student by scanning the code printed on that student's convocation. No typing is required.
FR-M4	The application shows immediately, after each scan, whether the student has been recorded, was already recorded, or was refused, and why.
FR-M5	The application returns to the scanning state immediately after showing the result, so that students can be processed one after another without any further action.
FR-M6	The proctor performs no administrative operation. The application cannot create, modify or delete anything.

Table 3.3: Rules enforced by the server

Ref.	Requirement
FR-S1	A code that was not produced by the system must be refused.
FR-S2	A student can be recorded at most once for a given examination session. Scanning the same student twice must not create a second record and must not be reported as an error.
FR-S3	A scan is accepted only while the examination is in progress, that is, from its scheduled start until its scheduled start plus its duration.
FR-S4	Every accepted scan is stored with the moment at which it was recorded.
FR-S5	The exported attendance of a session contains only the students recorded within the period of that session.
FR-S6	Every refusal must be reported with a reason that the proctor can act on.

Two of these rules deserve a word of explanation.

FR-S3 exists because attendance has a meaning only within the period of the examination. A record created an hour after the students have left the room does not describe anything that happened, and there is no legitimate reason to create one. Examinations at the faculty typically last an hour and a half, but the duration varies, which is why it is a property of each session rather than a fixed value.

FR-S2 distinguishes between two situations that are easily confused. A proctor who scans the same student twice has made no mistake worth reporting: perhaps they were unsure whether the first scan registered. The correct behaviour is to confirm that the student is present, without creating a second record and without displaying an error that would slow the queue.

3.2 Non-Functional Requirements

Table 3.4: Non-functional requirements

Property	Requirement
Concurrency	Several devices must be able to record attendance for the same session at the same time, without conflict and without a merging step afterwards.
Speed of use	Recording one student must require a single action. The result must appear at once, and be readable at a glance rather than read word by word.
Reliability	A failure must never be silent. If a scan is not recorded, the proctor must be told, immediately and unambiguously.
Security	Personal data and every operation that modifies data must be protected. Codes leave the institution on paper and must not be forgeable.
Simplicity of deployment	The system must run on ordinary equipment: a server, a browser, and telephones that the proctors already own.
Maintainability	The interface between the server and its clients must be documented precisely enough that a client can be written against it without reading the server's source code.

The requirement on reliability deserves emphasis, because it shaped several later decisions. In an examination room the proctor works quickly and cannot verify anything afterwards. An application that appears to have recorded a student who was not recorded is worse than an application that refuses openly, because the error is discovered only when the student is marked absent from an examination they sat.

3.3 Actors

The system has two human actors, and it is worth being precise about a third party who is not one.

The **administrator** is a member of the university staff responsible for organising examinations. At the faculty this is a single person. They create the examination sessions, obtain the codes to be printed, and retrieve the attendance afterwards. They are a permanent, identified member of staff.

The **proctor** supervises an examination room and records attendance. As explained in Chapter 1, proctors are frequently not permanent staff: master's and doctoral students are asked to help during periods when the number of examinations exceeds the number of available teachers. A proctor is present for a single session and may never use the system again.

The **student** is not an actor of the system. Students interact only with a printed document,

exactly as they do today. They install nothing, log in to nothing, and are not required to own a telephone. This is a deliberate property of the design and not an omission: telephones are not permitted in examination rooms.

The difference in status between the two actors has a direct consequence. The administrator performs operations that change data and must be authenticated. The proctor performs one operation that cannot be undone from the device and no operation at all that modifies the configuration of the system. Treating them as two different kinds of user, rather than as one user with two permission levels, keeps the mobile application free of credentials that a temporary helper would otherwise have to be given.

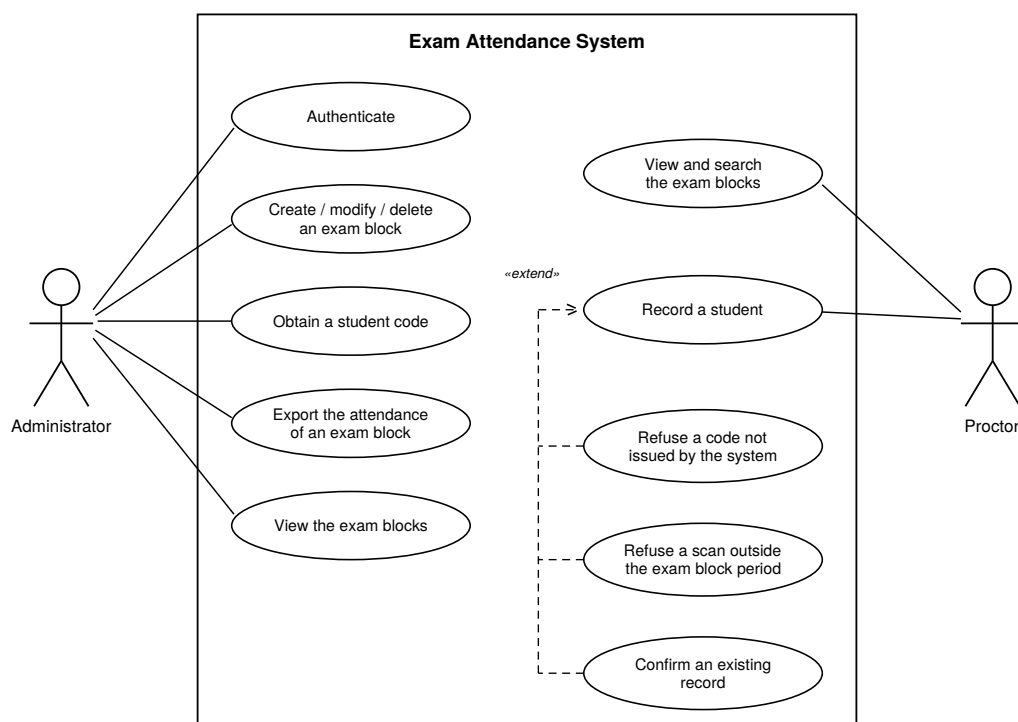


Figure 3.1: Use case diagram of the system

Figure 3.1 places the two actors on either side of the system. The administrator is connected to authentication, to the management of examination sessions, to the retrieval of a student code and to the export of attendance. The proctor is connected to consulting the list of sessions and to recording a student. The rules enforced by the server, listed in Table 3.3, appear as extensions of the recording use case rather than as separate cases, since no one initiates them directly.

3.4 Architecture

The system consists of three separate applications: a server, a mobile application used in the examination room, and a web interface used by the administrator.

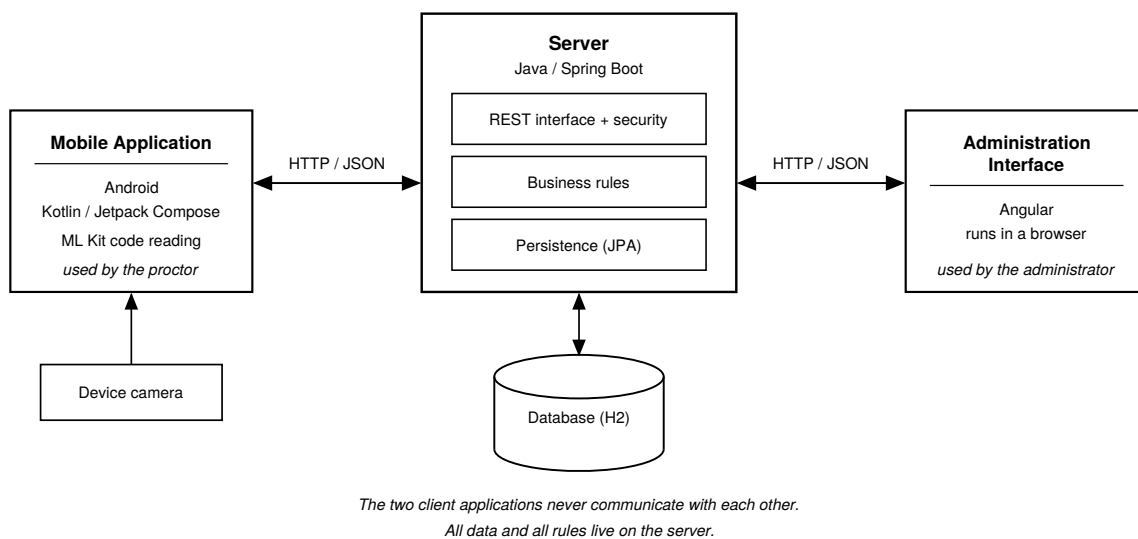


Figure 3.2: Architecture of the system

Figure 3.2 shows the server at the centre with the database beneath it, and the two client applications communicating with it over HTTP using JSON. The mobile application is connected to the camera of the device; the web interface runs in the administrator’s browser. No connection exists between the two clients.

The server holds all the data and enforces all the rules. Neither client decides whether a scan is valid, whether a student has already been recorded, or whether an examination is in progress. This is not a matter of convenience: it is the only arrangement in which several devices can record attendance simultaneously without their results having to be reconciled afterwards, and the only one in which the correctness of the result does not depend on the correctness of every device.

Placing all the rules on the server has a second consequence that matters in practice. A telephone has a clock that its owner can change, and that clock may simply be wrong. If the device decided whether an examination was still open, the answer would depend on a setting outside the institution’s control. The server has one clock, and it is the only one that counts.

The two clients are separate applications because they are used in different places, by different people, for different purposes. Combining them would mean giving a temporary proctor access to an interface that can delete examination sessions.

3.5 Technology Choices

Table 3.5: Technologies used and their role

Component	Technology	Role
Server	Java, Spring Boot	REST interface, business rules, persistence
Database	H2	Storage of the data model
Persistence	Spring Data JPA	Mapping between objects and tables
QR generation	ZXing	Production of the code images
Encryption	AES-GCM (Java)	Protection of the code payload
Authentication	JWT (jjwt), BCrypt	Administrator login, password storage
Mobile app	Kotlin, Jetpack Compose	Interface used in the examination room
QR reading	ML Kit Barcode Scanning	Detection and decoding from the camera
Mobile networking	Retrofit, Gson	Communication with the server
Administration	Angular, Bootstrap	Management of examination sessions

3.5.1 Server

Java with Spring Boot was chosen for the server. The framework provides, as part of a single coherent whole, the pieces this system needs: an HTTP layer for the REST interface, a persistence layer, input validation, and a security layer.

3.5.2 Database

H2 was used for the database. It requires no installation and no separate process, which made it appropriate for development and demonstration. It was configured to keep the data in memory, so the content is lost when the server stops.

This is a limitation, and it is stated as such in Chapter 5. It is worth noting what kind of limitation it is. Because the persistence layer maps objects to tables through JPA rather than through queries written for a particular database, moving to a database that stores data permanently is a change of configuration rather than a change of code. The schema is described by the entities, not by a script written for H2.

3.5.3 Mobile application

The mobile application is written in Kotlin with Jetpack Compose, and targets Android. A native application was preferred to a web page for two reasons. Access to the camera is direct

and reliable rather than mediated by a browser, and the application starts and responds without the delay of loading a page, which matters when a queue of students is waiting.

Android alone was targeted because it is what proctors at the faculty carry. Supporting a second platform would have doubled the client work for no gain within the time available.

ML Kit was used for reading the codes. It runs entirely on the device, requires no network call to detect a code, and handles the practical difficulties of reading from a live camera: focus, angle, and partial visibility.

3.5.4 Administration interface

The administration interface is an Angular application. It runs in a browser, so the administrator installs nothing, and it can be opened from any machine in the faculty. Angular provides form handling and validation as part of the framework, which is most of what this interface consists of. Bootstrap was used for presentation, in its stylesheet form only, without its JavaScript components.

3.5.5 Security choices

Chapter 2 presented sessions and tokens as the two ways of maintaining authentication. Tokens were chosen, in the form of JSON Web Tokens signed with HS256. The reason is the stateless character of the REST interface: sessions would have required the server to hold state for every logged-in administrator, which is precisely what that interface avoids. A symmetric algorithm is appropriate because the same server both issues and verifies the tokens; the asymmetric alternative solves a problem that does not arise here.

The consequence, also noted in Chapter 2, is that a token cannot be revoked before it expires. The expiry was therefore set to eight hours, which covers a working day without leaving a token valid indefinitely.

The administrator's password is stored as a BCrypt hash. The password itself is never stored and never recovered; verification hashes the submitted value and compares the results.

For the printed codes, authenticated encryption with AES in GCM mode was chosen over a signature. A signature would have proved that a code came from the system, which is what FR-S1 requires, but would have left the registration number readable by anyone who scanned the code with an ordinary telephone. Encryption satisfies FR-S1 and additionally keeps the identifier from being read by whoever picks up a lost convocation. GCM provides both the confidentiality and the integrity check in a single operation, and a fresh random initialisation vector is generated for every code, as the mode requires.

The distribution mechanism described in Chapter 2 makes key rotation inexpensive. Codes are printed for each examination session rather than issued once, so replacing the key affects only the codes printed after the change. This property was not a design goal; it follows from the decision to print the code on the convocation, and it is worth stating because a system based on

permanent cards would not have it. Figure 3.3 shows the proposed format of the convocation, with the code in place.

Convocation d'examen
Année Universitaire 2024/2025
Session Automne – Rattrapage | الدورة الخريفية – الاستدراكية



Nom	SABRI
Prénom	ZAKARIYA
Code APOGEE	
CIN/Passeport	
Filière	الأظمة المعلوماتية المضمنة



N° Exam.	Module	Semestre	Date	Horaire
A9-77	Anglais	S3	07/02/2025	11:15 – 12:00
FS12-5	Français	S3	07/02/2025	12:15 – 13:00

الوقت المطلوبة لإجبار الامتحان:

- على الطالب أن يكون مصحوبا بالوقت التالية: بطاقة التعريف الوطنية، وبطاقة الطالب أو عمل السجل الأصلي، وورقة الاستدعاء للامتحان.
- الطالب ملزم بالاملاء بالوقت المذكورة أعلاه متى طلب منه ذلك سواء داخل قاعة الامتحان أو خارجها.
- يمنع معه كليا حيازة أو استعمال أي وثائق أو مطبوعات أخرى غير مخصصة لها داخل قاعة الامتحان.

شروط خاصة بإجبار الامتحان:

- يجب على الطالب الحضور إلى قاعة الامتحان قبل الوقت المحدد، و الجلوس في المكان المخصص له ولا يمكنه التوجه لغير القاعة المحددة في الاستدعاء ولا اعتبار غائبا.
- يكتب على ورقة التحرير و المسودة الاسم الكامل، والرقم الوطني، ورقم التسجيل أوجي ورقم الامتحان للطالب وأي معلومات إضافية حسب الطلب.
- لا يسمح بمغادرة قاعة الامتحان قبل مرور نصف ساعة من انطلاق التقييم وعلى الطالب تسليم ورقة الأجوبة قبل المغادرة أو عند انتهاء الوقت المحدد.
- الاحتفاظ بالهاتف النقال أو استعمال أي وسيلة للاتصال أثناء التقييمات يعتبر غشا ويعاقب عليه طبقا للقوانين الجاري بها العمل.
- كل طالب مدعو للالتزام بالإجراءات التنظيمية للتقييمات واحترام جميع الساعرين على تطبيقها وحسن سريتها.
- عدم الاقترام بالتعليقات أوأله وكل حالة غش أو محاولة للتقييم بها أو المساعدة عليها تعرض صاحباها لعقوبات إدارية وزجرية مع إمكانية المتابعة القضائية.

Figure 3.3: The convocation, in the format proposed for this system, with the encrypted code printed in the upper right corner

3.6 Data Model

The data model was designed first and then presented to the supervisor for review before implementation. The review led to two changes, both of which are reflected in the model described here.

The first concerned modules. In the initial model, the module of an examination session was recorded as free text typed by the administrator. The review identified this as a weakness: two spellings of the same module name produce two apparently different examinations, and nothing prevents a typing error from creating a session that belongs to no real module. Modules were therefore made entities in their own right, grouped under the programme to which they belong, and an examination session now refers to one of them.

The second concerned the duration of an examination. Recording only the starting time is not sufficient to know whether an examination is in progress, which requirement FR-S3 needs, nor to determine which records belong to the session, which requirement FR-S5 needs. The duration was added as a property of each session.

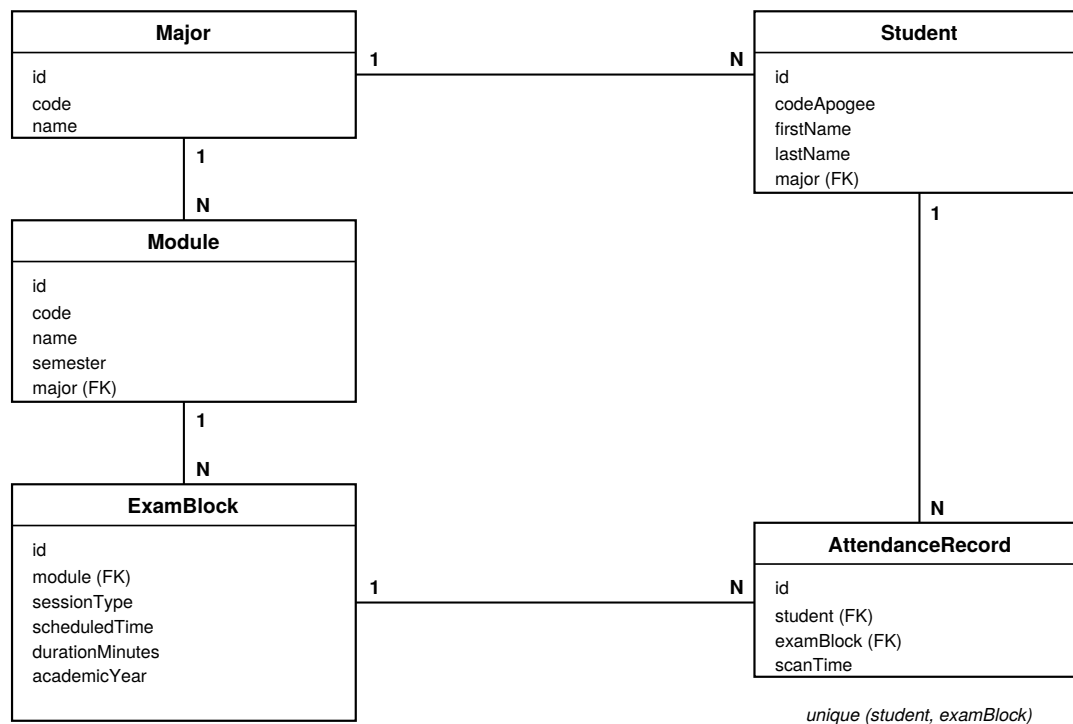


Figure 3.4: Entity relationship diagram of the system

The model shown in Figure 3.4 contains five entities. The entity representing one examination is named **ExamBlock** in the diagram and in the code; the text of this report refers to it as an examination session.

A **Major** represents a programme of study, identified by a code and a name.

A **Module** represents a taught subject, identified by a code, with a name, a semester, and the programme to which it belongs.

A **Student** holds the registration number that identifies the student throughout the system, the family name and given name, and the programme in which the student is enrolled.

An **ExamBlock** represents one examination in one room: the module concerned, the type of session, the scheduled starting time, the duration in minutes, and the academic year.

An **AttendanceRecord** links one student to one examination session, together with the moment at which the scan was performed.

Three properties of this model are deliberate.

The academic year is stored on the examination session as a value supplied by the administrator, rather than derived from the scheduled date.

The pair formed by a student and an examination session is unique. This is the enforcement of requirement FR-S2, and it is enforced by the database itself rather than by a check in the application, so that two devices scanning the same student at the same moment cannot both succeed.

Finally, all relationships are expressed in one direction only, from the dependent entity towards the one it refers to. An examination session refers to its module; a module does not hold a list of its examination sessions. Where the reverse relationship is needed, it is obtained by querying. This keeps the objects small and predictable, and avoids retrieving large quantities of related data as a side effect of reading a single record.

3.7 Recording a Scan

The last part of the design is the sequence of operations that takes place when a proctor scans a student.

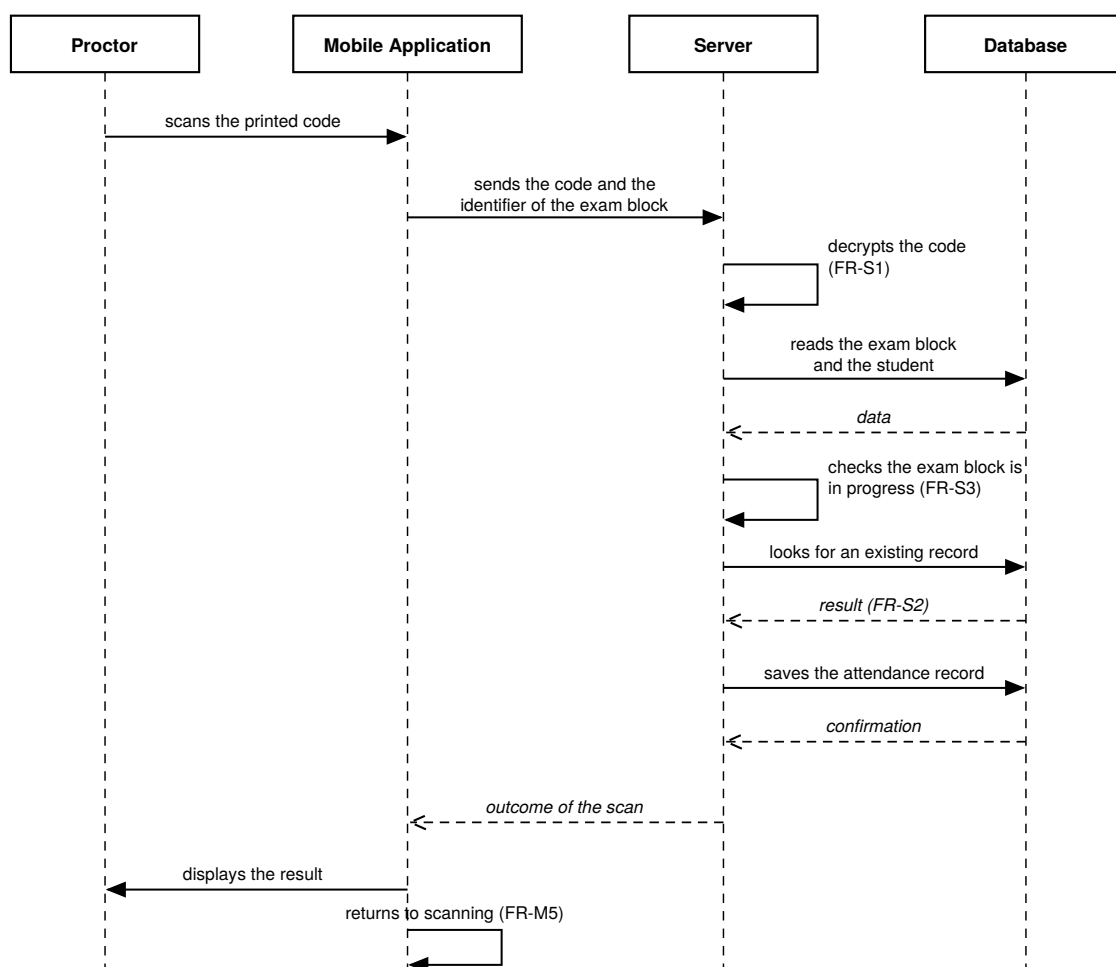


Figure 3.5: Sequence diagram: recording a student

Figure 3.5 shows that sequence, which proceeds as follows.

The proctor has already selected an examination session from the list, so the application knows which session the scan belongs to. The camera detects a code and decodes it into a text value. The application sends that value, together with the identifier of the selected session, to the server.

The server then performs a series of checks, in a fixed order. It decrypts the value; a value that was not produced by the system does not decrypt correctly and is refused, which satisfies FR-S1. It looks up the examination session and the student. It compares the current time against the scheduled start and the duration of the session, and refuses the scan if the examination has not begun or has already finished, which satisfies FR-S3. It then checks whether this student has already been recorded for this session; if so, it confirms the existing record rather than creating a second one, which satisfies FR-S2. Otherwise it creates the record with the current time and saves it.

The order of these checks is itself part of the design. The verification that the examination is in progress precedes the check for a duplicate, so that a second scan performed after the examination has ended is refused rather than reported as a confirmation of presence.

The server replies with a result that distinguishes the possible outcomes, and the application displays it. Whatever the outcome, the application returns immediately to the scanning state, as requirement FR-M5 demands.

3.8 Summary

The system is built around a server that holds the data and enforces every rule, with two independent clients: a native Android application for the examination room and a web interface for administration. Students are identified by an encrypted code printed on the convocation they already receive. The administrator authenticates with a token; the proctor does not authenticate at all, and cannot perform any operation that modifies the configuration of the system. The data model was reviewed with the supervisor before implementation, and represents modules as entities and the duration of an examination as an explicit property.

Chapter 4 describes how this design was implemented.

Implementation

This chapter describes how the design presented in Chapter 3 was built. It follows the same order: the server first, then the security mechanisms, then the two client applications, then the production of the attendance file. It continues with the second module of the internship, concerning the automatic reading of identity documents, and closes with the testing carried out.

The entity representing one examination is called an *exam block* in the code and in this chapter, which is the term used in the diagrams of Chapter 3.

4.1 The Server

4.1.1 Organisation of the code

The server is a Spring Boot application organised into packages by responsibility. The `model` package contains the entities that make up the data model. The `repository` package contains the interfaces through which those entities are read and written. The `service` package contains the business rules. The `controller` package contains the classes that receive HTTP requests. The `dto` package contains the objects exchanged with the clients, and the `exception` package the error types and the class that translates them into HTTP responses.

One package is organised differently. The classes concerned with security — the configuration of the filter chain, the filter that reads the token, the component that produces the response when a request is rejected, and the service that issues and verifies tokens — are grouped together in a `security` package rather than distributed among the others. These four classes have no meaning apart from one another, and grouping them makes the whole mechanism readable in one place.

4.1.2 Entities

The five entities of Figure 3.4 are mapped to tables by JPA. The relationships are declared in one direction only, from the dependent entity towards the one it refers to, as decided in Chapter 3. They are also declared as loaded on demand rather than automatically.

This second point deserves an explanation, because it is a choice that is easy to get wrong. When an attendance record is loaded, the student and the exam block it refers to are not loaded with it unless they are needed. Whether they are needed depends on the operation being performed, not on the entity itself: producing the attendance file requires the student's name, whereas recording a new scan does not. Rather than always loading them, the queries that need

the related data ask for it explicitly, in a single request to the database. This keeps the ordinary case cheap and makes the exceptional case explicit.

The uniqueness of the pair formed by a student and an exam block is declared as a constraint on the table rather than checked in the code. A check performed in the code can be defeated by two requests arriving at the same instant, since both may find no existing record before either has saved one. A constraint in the database cannot.

4.1.3 Objects exchanged with the clients

The clients do not receive the entities themselves. Each operation has its own object for what is sent and, where the two differ, another for what is returned.

The reason is visible in the case of an exam block. When the administrator creates one, the request contains the identifier of a module: a number. When the server returns an exam block, it contains the module's code and its name, because an interface has to display words rather than a number, and it also contains the identifier of the exam block itself, which the server assigned and the client could not have known. The two shapes are therefore genuinely different, and one class serving both would carry fields that are ignored in one direction and validation rules attached to fields that are never sent.

Returning the entity itself would also expose the internal structure of the data model to the clients, so that a change in the database would break them, and would risk sending related objects that nothing asked for.

4.1.4 Interface

Table 4.1: Operations exposed by the server

Method	Path	Purpose	Access
POST	/api/auth/login	Obtain a token	public
GET	/api/exam-blocks	List the exam blocks	public
POST	/api/exam-blocks	Create an exam block	protected
PUT	/api/exam-blocks/{id}	Modify an exam block	protected
DELETE	/api/exam-blocks/{id}	Delete an exam block	protected
GET	/api/exam-blocks/{id}/attendance/export	Download the attendance	protected
GET	/api/majors	List the programmes	protected
GET	/api/modules	List the modules	protected
GET	/api/students/{codeApogee}/qr-code	Obtain a student's code	protected
POST	/api/attendance/scan	Record a student	public

The operations available are listed in Table 4.1. Only two are open: reading the list of exam blocks and recording a student. Both are used by the mobile application, which has no account, for the reasons given in Section 4.2. Everything that modifies data, and everything that exposes personal data, requires authentication.

The list is deliberately short. An operation that returned the students already recorded for an exam block existed during development and was removed once it became clear that neither client used it. It exposed the same personal data as the attendance file while being open to anyone, so the system protected the file and left the list beside it unlocked. An operation nobody calls is still an operation an attacker can call, and one that no longer exists cannot be left unprotected by accident.

4.1.5 Validation and errors

Every object received from a client carries validation rules: a required module, a duration between one and one thousand four hundred and forty minutes, an academic year not below a plausible minimum. These rules are declared on the objects themselves and applied automatically before the request reaches the code that handles it.

Errors are translated into HTTP responses in one place, by a single class that handles every exception type the application can raise. This matters more than it may appear. Without it, each controller decides for itself what to return when something goes wrong, and the clients face a different error format depending on which operation failed. With it, every failure produces the same shape — an object containing a single field with a message — and the clients can be written once.

The status codes were chosen to distinguish situations that are genuinely different:

- **400** when the content of the request is wrong: a missing field, a value outside its permitted range, or a module identifier that matches no module. The address is correct; what was sent with it is not.
- **401** when a protected operation is attempted without a valid token.
- **404** when the thing addressed does not exist: an exam block identifier that matches nothing, or a code that decrypts to a registration number belonging to no student.
- **409** when the request is well formed and the address is correct, but the state of the system does not allow the operation: deleting an exam block that already has attendance records, or recording a student outside the period of the examination.

The distinction between 400 and 409 is not decoration. A client that receives 400 knows it sent something wrong and should correct it. A client that receives 409 knows it sent something valid that the system refuses at this moment, and the message explains why.

One case escapes this scheme. If the type of session is sent with a value other than the two the system recognises, the request fails while the JSON is being read, before validation runs, and produces a 500 rather than a 400. Neither client can send such a value, since both offer the two possibilities as a fixed choice, but the behaviour is wrong and is recorded among the limitations in Chapter 5.

4.1.6 Recording a scan

The scan operation implements the sequence of Figure 3.5. Its checks are performed in the order established in Chapter 3, and that order is part of the behaviour rather than an accident of how the code was written.

The value received is decrypted first. A code invented with an ordinary generator, a code altered after being issued, and a registration number typed in without encryption all fail at this step, because the authentication tag produced by the encryption mode does not verify. The response says only that the code is not valid, without saying which of these happened, since distinguishing them would help someone trying to construct a working code.

The decrypted value is then used to find the student, and the identifier sent with the request to find the exam block. A value that decrypts correctly but belongs to no student produces a different response from a value that failed to decrypt, which reveals to a caller that decryption succeeded. Reaching that point requires producing a value that decrypts, and that requires the key.

The current time is then compared against the scheduled start of the exam block and its duration. A scan before the start or after the end is refused with 409 and a message saying which of the two applies, together with the relevant time. The proctor is thereby told something they can act on, rather than that an unspecified error occurred.

Only then is the duplicate check performed. Placing it after the time check means that scanning a student a second time, after the examination has ended, is refused rather than reported as a confirmation that the student is present. The full order is therefore: decrypt the code, find the student, find the exam block, verify that the examination is in progress, check for an existing record, and only then save. If the student has already been recorded, the existing record is confirmed and no second record is created, and the response carries a different status code from that of a new record so that the application can show a different message.

4.2 Security

4.2.1 Protecting the printed code

The code printed on a convocation contains the student's registration number, encrypted with AES in Galois/Counter Mode, as decided in Chapter 3.

A fresh initialisation vector of twelve bytes is generated at random for each code. The vector, the ciphertext and the authentication tag produced by the mode are joined together and encoded in base64, giving a single text value that can be turned into a QR code. The vector is not secret and travels with the message, which is what the mode requires; what must never be repeated with the same key is its value, and generating it at random for every code satisfies that.

The key is read from the application's configuration and appears nowhere in the source code.

Decryption fails if any part of the value has been altered, because the mode verifies its authentication tag before returning anything. A code produced by an ordinary QR generator therefore fails at the first step, which is what requirement FR-S1 asks for.

4.2.2 Authenticating the administrator

The administrator logs in by sending a user name and a password. The password is compared against a BCrypt hash held in the configuration; the password itself is stored nowhere. If the comparison succeeds, the server returns a JSON Web Token signed with HS256 and valid for eight hours.

An incorrect user name and an incorrect password produce exactly the same response. A different message for each would tell someone attempting to guess which half they had already found.

Each subsequent request carries the token in the `Authorization` header. A filter placed before the application reads it, verifies the signature and the expiry, and records the identity of the caller. The filter does not reject anything: a request arriving without a token simply proceeds without an identity, and whether that is acceptable is decided by the authorisation rules. This separation matters, because the login operation itself must remain reachable without a token.

The authorisation rules are matched in order, and the first match decides. They are expressed by method and by path, so that reading the list of exam blocks stays open while creating, modifying and deleting one do not. This has a consequence worth keeping in mind: a rule that names one method leaves the same path open to every other method, so an operation added later under an existing path is not protected by the rules already written for it.

Sessions are disabled on the server, since the token carries the identity, and the protection against cross-site request forgery is disabled with them, as it applies to cookie-based authentication and not to a token sent explicitly in a header.

4.2.3 Why the mobile application has no account

The scan operation is open, and this was a decision rather than an omission.

Proctors are frequently master's or doctoral students helping for a single session. Giving them an account means either creating one for each of them, which nobody would maintain, or sharing one password among many temporary users, which would become public within a term and would open the administration interface, since it is the same account. An eight-hour token would also expire during a long examination period, and the application has no offline queue in which to hold scans while the proctor logs in again.

What protects the operation instead is the combination of the three rules already described. A scan requires a code that the system issued, which cannot be produced without the key. It is accepted only during the period of a specific examination. And it can record a given student only once. An attacker who obtained a valid code could mark that one student present

at an examination taking place at that moment — which requires them to hold the student's convocation, at which point the proctor's visual check is what stands in the way, exactly as it does today.

This is a weaker protection than authentication and it is presented as such in Chapter 5.

4.3 The Mobile Application

The application has two screens.

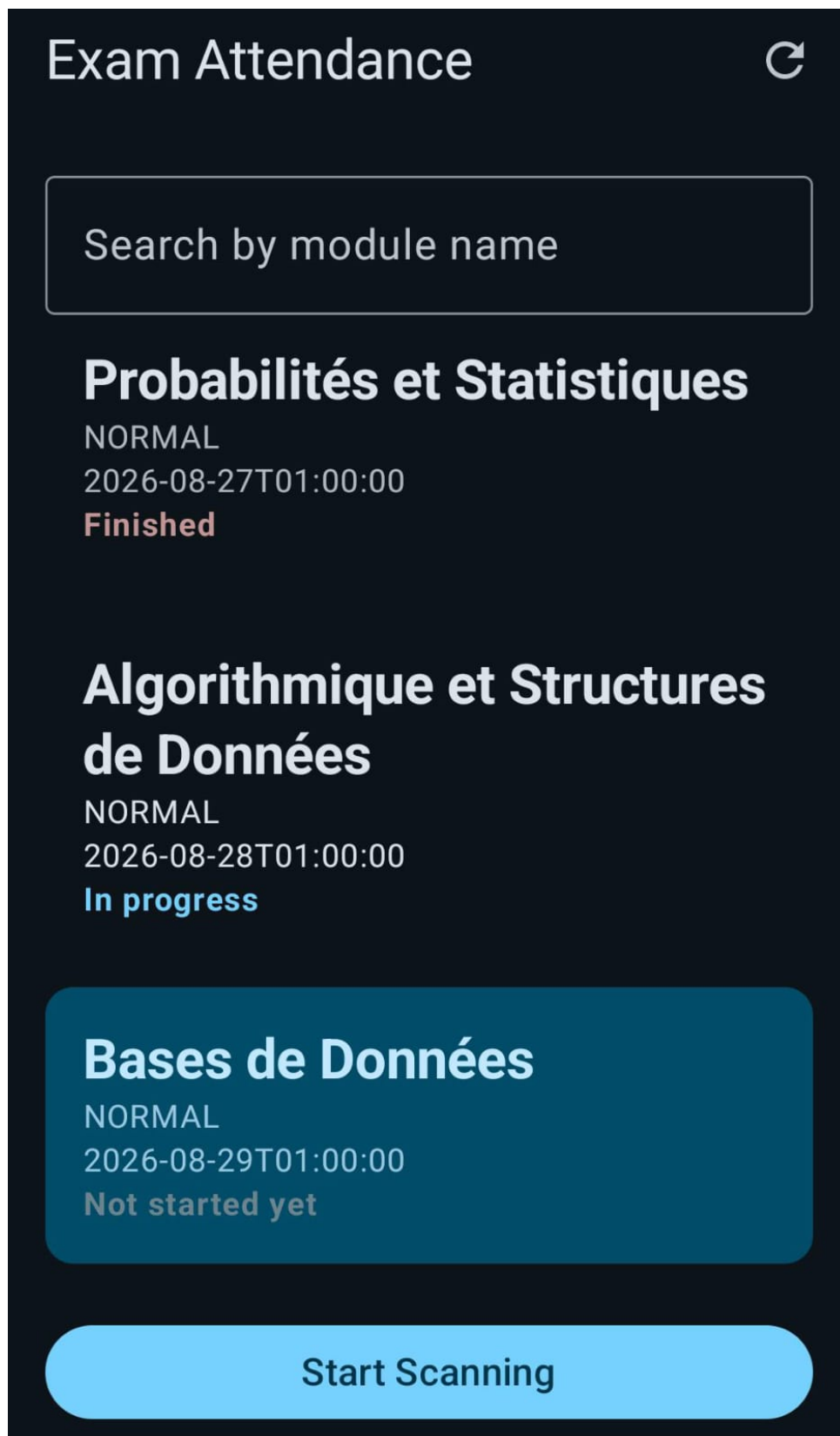


Figure 4.1: The exam block list on the proctor's telephone

4.3.1 Choosing an exam block

The first screen lists the exam blocks obtained from the server, with a search field that filters them by module name. The filtered list is recomputed from the full list each time the screen is drawn rather than stored separately, so that the two cannot fall out of step. A search that matches nothing shows a message rather than an empty screen, which would be indistinguishable from a failure to load.

Each entry shows whether the examination has not yet started, is in progress, or has already finished, using the same rule as the server. The entries that are not in progress are shown dimmed, and the state is written in words rather than indicated by shading alone.

Those entries nevertheless remain selectable, and this is deliberate. The state displayed is computed from the clock of the telephone, which its owner can change and which may simply be wrong. If a wrong clock could prevent a proctor from opening a genuine examination, a setting outside the institution's control would be able to stop an examination from being recorded. The label is therefore advice, not a rule. If the proctor selects a finished examination and scans, the server refuses and says why. The indication saves a wasted walk across the room; it does not decide anything.

The times sent by the server carry no time zone, and the application interprets them in the time zone of the device, on the assumption that the telephone is in the room where the examination is held. The consequence of that assumption being wrong is limited to the labels, for the reason just given.

A refresh control reloads the list, since the labels depend on the time and the list is a snapshot taken when the screen was opened. The same control serves to retry after a network failure.

4.3.2 Scanning

The second screen shows the camera. A detected code is sent to the server together with the identifier of the chosen exam block, and the outcome is displayed at once.

The application distinguishes several outcomes: the student has been recorded, the student was already recorded, the scan was refused by the server, the server could not be reached, and there is no network connection. A recorded student and a refused scan are shown in visibly different ways, not only in different words, because a proctor working through a queue reads the colour before the text. Confusing a refusal with a success is the most damaging error this screen could make: the student would leave believing they had been recorded, and would be marked absent from an examination they sat.

When the server refuses a scan, the application displays the message the server sent rather than a message of its own. It reads that message from the response for any refusal, whatever the reason, instead of holding one text per case. The server therefore decides both what is refused and how the refusal is explained, and a new reason added later requires no change to the application. If no message can be read from the response, a general message is shown; the

proctor is never left with a blank screen.

After any outcome, the screen returns immediately to the camera, so that students can be processed one after another without touching the device.

There is no offline queue. If the network is unavailable the scan fails and says so. Storing scans on the device to send later would mean that a proctor could believe a room had been recorded when nothing had reached the server, and that duplicate detection — which requires the server — could not be performed at the moment of scanning.

4.4 The Administration Interface

Module	Session	Scheduled time	Duration	Year	Actions		
Probabilités et Statistiques	NORMAL	2026-08-27T01:00:00	90 min	2027	Edit	Delete	Download Attendance
Algorithmique et Structures de Données	NORMAL	2026-08-28T01:00:00	90 min	2027	Edit	Delete	Download Attendance
Bases de Données	NORMAL	2026-08-29T01:00:00	90 min	2027	Edit	Delete	Download Attendance
Résistance des Matériaux	NORMAL	2026-08-30T04:00:00	90 min	2027	Edit	Delete	Download Attendance

Figure 4.2: The list of exam blocks in the administration interface

The interface opens on a login page. Once the administrator has logged in, the token is kept for the duration of the browser tab and is cleared when the tab is closed. It is attached to every request by a single component placed between the application and the network, rather than by each part of the application separately.

The interface never inspects the token and never checks whether it has expired. A browser's clock can be wrong, and a second implementation of the same rule can only disagree with the first. When the server answers that a token is not valid, the interface returns to the login page. That is the only response treated in this way: a refusal caused by the state of the data leaves the administrator where they are, with the message displayed.

The main screen lists the exam blocks and allows one to be created, modified or deleted. The module is chosen from a list loaded from the server, showing the code and the name of each module, and the value sent is the module's identifier. The list is loaded once when the screen opens, and if it cannot be loaded the form does not open at all, since a form whose only required choice is empty can never be submitted successfully.

The fields are validated in the browser using the same limits and the same messages as the server. This duplication is deliberate and its purpose is different on each side: in the browser it gives the administrator an immediate answer, and on the server it guarantees that invalid data cannot be stored regardless of what the browser did.

An invalid form does not submit, and it says why. Every field that fails is marked at the

same moment, together with a line above the button. This is worth stating because the opposite behaviour — a button that does nothing — is a common and damaging outcome: the person cannot tell whether the application is broken, the network has failed, or they have made a mistake.

Deleting an exam block that already has attendance records is refused by the server, which returns the number of records concerned. The interface shows that message beside the row and offers, in place of the delete button, a choice between confirming and cancelling. Confirming repeats the request with an explicit instruction to delete the records as well. The confirmation appears within the table rather than in a separate window, which keeps the administrator's attention on the row concerned.

4.5 Producing the Attendance File

The attendance of an exam block is downloaded as a CSV file, which can be opened in a spreadsheet, printed, or kept as it is.

The operation takes no parameters other than the identifier of the exam block. Everything else it needs — the scheduled start and the duration — is a property of that exam block. An earlier arrangement in which the duration was supplied with the request would have allowed two downloads of the same examination to produce different files.

The file contains the students recorded within the period of the examination. Since the server already refuses scans outside that period, this filter should never remove anything: it is a second barrier rather than a working filter, and if it ever removed a row it would indicate that something had gone wrong elsewhere. Retaining it costs nothing and guarantees the property stated in requirement FR-S5.

Two details concern the file rather than the data.

Names at the faculty contain accented characters. A CSV file opened in a spreadsheet is interpreted according to whatever encoding the spreadsheet assumes, and a common spreadsheet assumes an encoding in which those characters are displayed incorrectly. The file therefore begins with a byte order mark, a short sequence that identifies the encoding explicitly. It is a small detail, and without it the names in the file are wrong.

The name of the file is built by the server from the module, the type of session and the academic year, and transmitted in the response header. The browser reads it from there rather than inventing one, so that a file downloaded on any machine carries the same name.

An examination at which nobody was recorded produces a valid file containing only the header row. An empty file and an error are different things, and a system that reported an error would leave the administrator unable to distinguish between an examination nobody attended and an examination whose data could not be read.

4.6 The Verification Module

The second module of the internship concerns the verification of the data a student enters at administrative registration, by reading their identity document.

4.6.1 What was built

The module is complete as a pipeline. A student registers by entering their personal details, then uploads a photograph of their national identity card and, separately, of their baccalaureate certificate. Each document is read by an optical character recognition engine running on the server. The raw text produced is parsed into fields. Each field is compared with what the student entered. The outcome of the comparison is stored, so that it can be consulted afterwards rather than only displayed once.

Two documents are handled, and they required different treatment. The parser for each was written against real photographs.

4.6.2 What the documents showed

The identity card and the baccalaureate certificate fail in different ways, and the differences are instructive.

On the identity card, the labels printed beside each field rarely survive recognition. The parser cannot therefore look for a label and read what follows it, and has to rely on the position and the shape of the values: dates are recognised by their pattern, and the two lines in capital Latin letters near the top of the text are the given name followed by the family name. That last point was established by testing against several cards; the opposite order had been assumed at first and was wrong.

On the baccalaureate certificate the labels do survive, which allows the parser to anchor directly on them. The separator used in dates also differs between the two documents, so both forms have to be accepted.

Three practical findings emerged that no description of the technology had suggested. A photograph taken sideways produces meaningless output, with no indication that anything is wrong, because the engine does not correct orientation by itself. Configuring the engine to treat the page as a single block of text, rather than attempting to analyse its layout, transformed the result on the busier of the two documents. And one particular sample, with a heavily patterned background, remained unreadable under every configuration tried, and was kept deliberately as a test case for the situation where recognition simply fails.

4.6.3 Three outcomes rather than two

The most important consequence of these findings concerns the comparison itself.

The registration number on the identity card is the least reliable field: sometimes it is read perfectly, sometimes it comes back as an unusable sequence of characters. The place of birth is often unrecoverable. Dates are generally reliable but a single digit can be misread.

A comparison that reports only agreement or disagreement would therefore tell a student that their registration number does not match, when the truth is that the camera could not read it. That is not a small imprecision: it is an accusation of error where none has been shown.

The comparison therefore produces one of three outcomes for each field: the values agree, the values differ, or the field could not be read with enough confidence to say anything. The third outcome is not a failure of the system but a statement about the evidence, and it is the outcome that makes the module usable at all.

4.6.4 Why a more accurate engine was not used

Commercial recognition services perform considerably better on photographs than an engine installed locally. Using one requires sending the image to a third party.

The image is a student's identity document. Transmitting it outside the institution, to a service whose processing and retention cannot be verified, is not a decision that can be taken for convenience. The constraint is not technical and no gain in accuracy removes it. It confines the realistic options to engines that run on the institution's own machines, and those are the ones whose limitations are described above.

4.6.5 What the module is for

Taken together, these results support a conclusion that is narrower than the original ambition but is a genuine result. Automatic reading of these documents cannot decide whether a student's declared information is correct. It can read part of it, part of the time, and it can say when it has read nothing.

That is still useful. An operator checking registrations can be shown which fields agree, which differ and which could not be read, and can concentrate on the second category instead of comparing every field of every document by eye. The module is an aid to a human verifier, not a replacement for one, and it is honest about which of the two it is.

4.7 Testing

Testing was manual and was carried out continuously, as described in Chapter 1. Each rule was exercised deliberately, including in the cases where it was expected to refuse, and each part of the system was tested first on its own and then through the application that uses it.

The parsing of the identity documents was the exception: because it is a matter of pure text processing with no dependence on the rest of the system, it was covered by automated tests running against real recognition output.

Table 4.2: Representative verifications carried out

Situation	Expected and observed behaviour
A student is scanned during an examination in progress	Recorded, confirmation displayed
The same student is scanned again	Confirmed as already present, no second record
A student is scanned before the examination begins	Refused, message stating the start time
A student is scanned after the examination has ended	Refused, message stating the end time
A code produced by an ordinary QR generator is scanned	Refused as not valid
A protected operation is attempted with no token	Refused, login required
A token whose contents have been altered is presented	Refused
An exam block with attendance records is deleted	Refused, number of records reported
The same deletion is confirmed explicitly	Exam block and records deleted
The server is stopped and a scan is attempted	Clear message, no silent failure
An examination with no attendance is exported	Valid file containing only the header row
The clock of the telephone is changed and a scan attempted	Server accepts or refuses according to its own clock

Table 4.2 lists representative cases. The last one is worth singling out. Changing the clock of the device alters the labels shown in the list, since those are computed on the telephone, but has no effect on whether a scan is accepted, since that is decided by the server. This was verified deliberately rather than assumed, and it is the clearest demonstration that the correctness of the record does not depend on the device.

4.8 Summary

The system was built as designed: a server holding the data and enforcing every rule, a mobile application that scans and displays what the server decides, and an administration interface for managing examinations and retrieving their attendance. The printed codes are protected by authenticated encryption, the administrative operations by an authentication token, and the recording of a scan by three rules that together make a forged or misplaced scan difficult without requiring proctors to hold accounts.

The verification module produced a narrower result than intended, and one that is stated plainly: recognition of these documents is an aid to a human verifier and not a substitute for one.

Chapter 5 sets out what the system achieves, what it does not, and what would be required

before it could be used in production.

Results, Limitations and Future Work

This chapter states what the system does, what it does not, and what would be needed before it could be used at the faculty. The limitations are separated into three kinds, because they are not equivalent: some are deliberate boundaries of what the system was meant to do, some follow from the conditions in which it was built, and some are simply defects.

5.1 What the System Does

The attendance system is complete and works end to end. An administrator logs in, creates an exam block by choosing a module and setting the time, the duration and the academic year, and obtains the code to be printed for a student. A proctor opens the mobile application, selects an exam block from the list, and records students by scanning the code on each convocation. The administrator then downloads the attendance of that exam block as a file.

Measured against the objectives stated in Section 1.4, the results are as follows.

Attendance can be recorded by several people at the same time. All data and all rules are held by the server, so any number of telephones can record students for the same exam block simultaneously. The uniqueness of a student within an exam block is guaranteed by the database rather than by the devices, so two proctors scanning the same student at the same instant cannot produce two records. This was the central objective, and it is the one the paper procedure could not satisfy at all.

A student is recorded without their name being searched for. The proctor performs one action, and the seating order of the room has no effect on how long it takes.

The verification of identity is unchanged. The proctor still compares the face with the photograph on the convocation. The system was deliberately built not to touch this step.

A record is produced that the administration can keep. The attendance file opens in a spreadsheet, displays accented names correctly, carries a name built from the module and the session, and is a valid file even when nobody was recorded.

The data and the operations that modify it are protected. Codes cannot be forged without the encryption key. Every operation that changes data or exposes personal data requires an authentication token. The administrator's password is stored only as a hash.

Two further results are worth stating because they were not part of the original objectives.

The distribution of the codes costs nothing. Because the code is printed on a document the administration already produces for every student before every examination, no cards have to be issued, no replacements have to be handled, and replacing the encryption key affects only the documents printed after the change.

The correctness of the record does not depend on the telephone. The device's clock and its time zone affect only the labels displayed in the list, never whether a scan is accepted. This was verified deliberately, by altering both and confirming that the server's decision was unchanged.

5.2 Limitations

5.2.1 Deliberate boundaries

These are not defects. They are things the system was designed not to do, and the reasons are given in the chapters where the decisions were made.

The system does not verify that the person presenting a convocation is the student named on it. That check is performed by the proctor, as it always has been, and the system replaces the search through the list rather than the verification of identity.

The mobile application does not authenticate. Proctors are frequently master's or doctoral students helping for a single session, and an account for each of them would not be maintained while a shared account would become public and would also open the administration interface. The scan operation is protected instead by the combination of three rules: the code must have been issued by the system, the examination must be in progress, and a student can be recorded only once.

The mobile application does not work offline. A scan that cannot reach the server fails and says so. Storing scans on the device to be sent later would allow a proctor to believe a room had been recorded when nothing had arrived, and would make duplicate detection impossible at the moment of scanning.

A scan is refused strictly outside the period of the examination, with no tolerance before the scheduled start. The administrator controls that time, so an examination at which scanning should begin ten minutes early can simply be scheduled ten minutes early.

5.2.2 Limits imposed by the conditions of the internship

These follow from the environment rather than from the design, and each would be removed by a change outside the code.

The data is held in memory and is lost when the server stops. Access to real student records was not authorised, so there was no production database to connect to and no reason to install one. The persistence layer maps objects to tables rather than issuing queries written for a particular database, so moving to permanent storage is a change of configuration.

For the same reason, the student table contains representative rather than real data, and the programmes and modules are loaded at startup rather than created through an interface. The application reads from a student table; replacing that table with the university's own is a change of data source, not a change of code.

The system communicates over HTTP rather than HTTPS. Everything runs on a local network for development and demonstration. Moving to HTTPS requires a certificate, a change to the server configuration, an update to the address used by each client, and the certificate to be trusted on the Android device and in the browser. None of this is difficult; none of it was appropriate before a decision to deploy.

There is one administrator account, defined in the configuration file. There is no user table, no second account, and no notion of roles. At the faculty a single person creates the examinations, so a second role would have had no occupant.

The system has never been used during a real examination. Every rule has been exercised deliberately, but a room containing sixty students, several proctors, imperfect lighting and folded documents will reveal things that testing cannot. This is the most significant limitation in the chapter.

5.2.3 Defects and rough edges

These are genuine faults, found during testing and documentation, and left in place because they were judged less urgent than the work described elsewhere.

If the type of session is sent with a value other than the two the system recognises, the request fails while the JSON is being read, before validation runs, and the server answers with an internal error rather than reporting an invalid value. Neither client can produce such a request, since both present the two possibilities as a fixed choice, but the behaviour is wrong.

A scan whose code decrypts successfully but corresponds to no student produces a different response from a code that failed to decrypt, and the response repeats the decrypted value. A caller can therefore learn whether decryption succeeded. Reaching that point requires producing a value that decrypts, which requires the encryption key, so the disclosure is small, but it exists.

The message reported when an exam block cannot be found is worded in three different ways depending on which operation produced it. A client that reacts to the text of the message rather than to the status code would behave inconsistently.

Deleting an exam block together with its attendance records is requested by adding a single value to an ordinary delete request. The administration interface asks for confirmation before doing so, but the server does not, and the records cannot be recovered afterwards. For an operation that destroys attendance data, that is thin protection.

The operation that produces a student's code does not check that the registration number belongs to a real student. It returns a valid code for any value. The consequence is limited, since such a code is refused at scan time, but the operation is more permissive than it needs to be.

The state shown beside each exam block in the mobile application is computed when the list is loaded and does not update by itself. A block that finishes while the screen is open continues to be shown as in progress until the list is refreshed. The label is advisory and the server refuses the scan regardless, so the consequence is a wasted attempt rather than a wrong record.

Times are stored and transmitted without a time zone. The mobile application interprets them in the time zone of the device, on the assumption that the telephone is in the room where the examination takes place. If that assumption were false the labels would be wrong, though the records would not.

5.2.4 The verification module

The limitations of the second module are of a different kind, and are described in Section 4.6. In summary: the engine does not correct the orientation of an image, so a photograph taken sideways produces meaningless output with no indication that anything is wrong; the registration number and the place of birth are the least reliable fields on the identity card; and one test document remained unreadable under every configuration attempted.

The conclusion drawn from this is not that the module failed but that the task is harder than it appears. Automatic reading of these documents can support a human verifier by showing which fields agree, which differ and which could not be read. It cannot decide.

5.3 Future Work

Table 5.1: Work required before the system could be used

Order	Work
1	Add the encrypted code to the convocation template, in cooperation with the department that produces it. Nothing else can be tested in real conditions until this is done.
2	Connect the system to the university's own student, programme and module data in place of the loaded data.
3	Move the database to permanent storage.
4	Serve the system over HTTPS, with a certificate trusted by the browsers and the devices that use it.
5	Run a pilot during a real examination, with the paper sheet kept in parallel as a fallback and as a means of comparison.
6	Correct the defects listed in Section 5.2.3.

Beyond that list, three directions would extend the system rather than complete it.

The mobile application could be given a form of identification that does not require a personal account: a key held by the application and issued to a device rather than to a person, or an enrolment step performed once by the administrator for each telephone used. This would close the gap described in Section 5.2.1 without giving temporary proctors credentials that also open the administration interface.

The system records who was present but not who recorded them. Storing the device or the proctor responsible for each scan would make it possible to answer questions about a disputed

record, which paper answers today through a signature.

The verification module could be reconsidered if the university obtained access to a recognition service under conditions that satisfy the protection of personal data, whether through an agreement with a provider or through a service hosted by the institution itself. The constraint that shaped that module is legal rather than technical, and it is the constraint that would have to change first.

5.4 Summary

The attendance system meets the objectives set at the start of the internship. It removes the dependence on a single shared document, it records a student without a search through a list, it leaves the verification of identity with the proctor, and it produces a file the administration can keep. Its codes cannot be forged without the key, and its administrative operations require authentication.

It has not been used in a real examination, it holds its data in memory, and it communicates over an unencrypted connection. These are the consequences of building a system rather than deploying one, and each is a known quantity with a known remedy.

The verification module produced a narrower result: a working pipeline whose accuracy is limited by the documents it must read and by the impossibility of sending those documents elsewhere. It is honest about being an aid to a human verifier rather than a replacement for one, and that honesty is built into its design, in the distinction it makes between a field that disagrees and a field it could not read.

General Conclusion

The problem this internship set out to solve was not a shortage of technology. It was that a single sheet of paper had become the only place where the attendance of an examination could be recorded. Everything difficult about the task followed from that: one person had to do the work however many proctors stood in the room, and when students did not sit in their assigned places, that one person had to search a list of several dozen names for each of them.

The system built during these two months removes that constraint. Attendance is held by a server, so any number of telephones can record students for the same examination at once, and the database itself guarantees that a student cannot be recorded twice. Students are identified by a code printed on the convocation they already receive before every session, so nothing has to be issued, replaced or carried. The proctor performs one action per student and sees the result immediately. At the end, the administration downloads a file it can print, keep or archive.

Two decisions shaped the result more than the others.

The first was to place every rule on the server and none on the devices. A telephone's clock can be wrong, its copy of the list can be out of date, and it may be held by someone who has never used the application before. None of that can affect a stored record, because the device only asks and displays; the server decides. This was verified deliberately, by altering the clock and the time zone of the telephone and confirming that the answer did not change. It is the property that makes the system trustworthy with people who are not trained on it.

The second was to be precise about what the system replaces. The proctor still compares a face with a photograph, exactly as before. Removing that step was never the goal, and saying so plainly is not a weakness of the design but a statement of where its boundary lies. What the system replaces is the search through a list, and the paper sheet that made the search unavoidable.

The second module of the internship, the automatic verification of registration data, produced a different kind of result. The pipeline works: a document is read, its fields are extracted, they are compared with what the student typed, and the outcome is stored. What is unreliable is the reading of a photograph of an identity card, and the services that would read it more accurately cannot be used, because sending a student's identity document to a third party is not a decision a university can take for convenience. The useful conclusion is therefore narrower than the original ambition and more honest than an accuracy figure: recognition of these documents can tell an operator which fields agree, which differ and which could not be read, and the distinction between the last two is the whole value of the module. It assists a human verifier; it does not replace one.

The most important limitation is stated plainly in Chapter 5: the system has never been used during a real examination. Every rule has been exercised on purpose, including in the cases where it should refuse, but a room with sixty students, several proctors, uneven lighting and

folded papers will show things that deliberate testing cannot. The first step towards using the system is not more code; it is adding the printed code to the convocation, which depends on the department that produces that document, and then running one examination with the paper sheet kept alongside as a comparison.

On a personal level, this internship covered ground I had studied but not applied. Building a server, a native mobile application and a web interface that all had to agree on the same interface taught me why that interface has to be written down before the clients are built, and not after. The security work followed the same lesson: authenticated encryption for the printed codes, a signed token for the administrative operations, and hashed credentials were not additions made at the end, but decisions taken while the server was being designed.

What I found harder, and more valuable, was the judgement rather than the code. Deciding that the mobile application should have no account, that a finished examination should still be selectable, that an empty attendance file is a valid answer and not an error, or that a comparison needs three outcomes and not two: none of these was a technical difficulty, and each of them changes what happens to a student. Working with a supervisor who reviewed the design before it was built, and whose review changed it, was part of the same lesson. A system is judged by how it behaves when something is wrong, and most of the decisions that matter are made long before anything goes wrong.

Bibliography

- [1] International Organization for Standardization. *ISO/IEC 18004:2015 — Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*. ISO, Geneva, 2015.
- [2] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000.
- [3] M. Jones, J. Bradley and N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force, May 2015.
- [4] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. FIPS PUB 197, November 2001.
- [5] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D, November 2007.
- [6] N. Provos and D. Mazières. A Future-Adaptable Password Scheme. In *Proceedings of the USENIX Annual Technical Conference*, 1999.
- [7] Tesseract OCR. *Tesseract documentation*. <https://tesseract-ocr.github.io/> (accessed August 2026).
- [8] Spring. *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/> (accessed August 2026).
- [9] Spring. *Spring Security Reference Documentation*. <https://docs.spring.io/spring-security/reference/> (accessed August 2026).
- [10] Java JWT. *jjwt — JSON Web Token for Java*. <https://github.com/jwt/jjwt> (accessed August 2026).
- [11] ZXing. *ZXing — Multi-format 1D/2D barcode image processing library*. <https://github.com/zxing/zxing> (accessed August 2026).
- [12] H2 Database Engine. *H2 documentation*. <https://www.h2database.com/> (accessed August 2026).
- [13] Android Developers. *Jetpack Compose*. <https://developer.android.com/jetpack/compose> (accessed August 2026).

- [14] Google Developers. *ML Kit — Barcode Scanning*. <https://developers.google.com/ml-kit/vision/barcode-scanning> (accessed August 2026).
- [15] Angular. *Angular documentation*. <https://angular.dev/> (accessed August 2026).
- [16] Bootstrap. *Bootstrap 5 documentation*. <https://getbootstrap.com/docs/5.3/> (accessed August 2026).

Interface Contract

This appendix reproduces the contract between the server and the two client applications. It was written before the clients were built and updated whenever the interface changed, so that neither client depended on reading the server's source code. It is included here because the maintainability requirement stated in Section 3.2 depends on the interface being written down rather than inferred.

All paths are relative to the server root. All request and response bodies are JSON unless stated otherwise. Times are ISO-8601 local date-times without a timezone offset.

A.1 Operations

Ten operations are exposed. Three are public: listing the exam blocks, logging in, and recording a scan. Everything else requires an authenticated administrator.

Table A.1: The complete set of operations

Method	Path	Purpose	Access
POST	/api/auth/login	Obtain a token	public
GET	/api/exam-blocks	List the exam blocks	public
POST	/api/exam-blocks	Create an exam block	protected
PUT	/api/exam-blocks/{id}	Modify an exam block	protected
DELETE	/api/exam-blocks/{id}	Delete an exam block	protected
GET	/api/exam-blocks/{id}/attendance/export	Download the attendance	protected
GET	/api/majors	List the programmes	protected
GET	/api/modules	List the modules	protected
GET	/api/students/{codeApogee}/qr-code	Obtain a student's code	protected
POST	/api/attendance/scan	Record a student	public

A.2 Authentication

A token is obtained from POST /api/auth/login and sent on protected requests in a single header:

Authorization: Bearer <token>

The literal word Bearer, one space, then the token. The token is a JSON Web Token signed with HMAC-SHA256 and valid for eight hours; the login response carries an `expiresAt` field giving the exact instant it stops being accepted. There is no refresh operation — an expired token is replaced by logging in again.

The authorisation rules are evaluated in the order below, first match winning, and matching is by path *and* method.

1. OPTIONS /** — public, for CORS preflight
2. GET /api/exam-blocks/*/attendance/export — token required
3. GET /api/majors/** — token required
4. GET /api/modules/** — token required
5. GET /api/students/*/qr-code — token required
6. POST /api/exam-blocks/** — token required
7. PUT /api/exam-blocks/** — token required
8. DELETE /api/exam-blocks/** — token required
9. everything else — public

That final rule covers exactly three operations: listing the exam blocks, logging in, and scanning. Nothing else in the interface is reachable without a token. The database console is disabled outright rather than left to these rules.

A missing header, an expired token and a tampered signature all produce the same response. Nothing in the reply distinguishes them, because telling a caller *which* of the three failed is information they have not earned.

A.3 Error Format

Every operation uses one error shape, selected by the HTTP status code.

Table A.2: Error responses

Status	Meaning and body
400	The body failed validation, or a printed code could not be decrypted. Validation failures return a map of field name to message. An undecryptable code returns a single message instead.
401	Wrong credentials, or a missing, invalid or expired token.
404	The requested student or exam block does not exist.
409	The request conflicts with the current state of the data and was refused.
500	An unexpected server condition, or a body that could not be parsed. No internal detail is ever returned.

A successful response never carries an error key. Operations that return a list answer with an empty array and status 200 when there is nothing to return; an empty result is not an error.

A.4 Obtaining a Printed Code

GET `/api/students/{codeApogee}/qr-code` returns a PNG image, 300 by 300 pixels, encoding the *encrypted* registration number. The raw number is never encoded. A new random initialisation vector is used on every call, so two codes generated for the same student look different and both decrypt to the same value.

The operation does not check that the registration number belongs to a real student. It always answers with an image. An unknown number only fails later, at scan time, which keeps code generation independent of the student table.

A.5 Recording a Scan

POST `/api/attendance/scan` takes an encrypted code and an exam block identifier:

```
{ "codeApogee": "<encrypted value>", "examBlockId": 1 }
```

The request carries no time value of any kind, and the server would not read one if it did. Every decision about whether the examination is running is taken from the server's own clock.

The checks run in a fixed order: decrypt the code, find the student, find the exam block, test whether the block is in progress, test whether this is a repeat, then save. The in-progress test runs *before* the duplicate test, so a second scan against a block that has since finished is refused for being out of the window rather than reported as already present.

A successful reply carries the decrypted registration number, the student's name and the time of the scan. Status 201 means a new record was created; status 200 means the student was already present and the time returned is the original one. The body is identical in both cases — only the status distinguishes them.

Table A.3: Refusals when recording a scan

Status	Cause
400	The code could not be decrypted: malformed, altered, or a plain value that was never encrypted by the system.
404	The code decrypted correctly but matches no known student.
404	The exam block identifier matches no exam block.
409	The server's clock says the examination has not started. The message names the start time.
409	The server's clock says the examination has finished. The message names the end time.

A.6 Creating an Exam Block

POST `/api/exam-blocks` takes a module identifier, a session type, a start time, a duration and an academic year:

```
{
  "moduleId": 1,
  "sessionType": "NORMAL",
  "scheduledTime": "2026-09-15T09:00:00",
  "durationMinutes": 90,
  "year": 2026
}
```

Three points in this body were decided deliberately.

The client sends a module *identifier*, not a module name. The catalogue is read from the server, so a client cannot invent a module by spelling one.

The duration is required. It is held server-side in a type that can be absent, so that omitting it is refused rather than silently treated as zero. A block with no duration would be a block that never ends.

The academic year is independent of the calendar year in the start time and is not derived from it. A session held in September may belong to the year that began the previous autumn.

The session type must be exactly `NORMAL` or `RATTRAPAGE`.

A.7 Downloading the Attendance

GET `/api/exam-blocks/{id}/attendance/export` returns a CSV file rather than JSON. The file name is built by the server from the module, the session type and the academic year, so that two files downloaded on different days for the same examination carry the same name.

The file begins with a byte-order mark, without which accented characters in Moroccan names are displayed wrongly by common spreadsheet software.

The export applies the same time window as the scan. Because a scan outside the window is already refused, this filter should never remove a row; it is a second barrier rather than a first. An examination at which nobody was recorded produces a valid file containing only the header row — not an error, because an error would leave the administration unable to distinguish an empty room from a failure to read the data.